Lab4:组合逻辑电路 实验报告

PB24111621 肖瀚

一、实验目的与内容

1. 学习组合逻辑电路的简单设计流程，从列真值表到卡诺图化简、表达式化简，再到最后的代码实现；
2. 了解简单的组合逻辑元件，包括编码器、译码器以及加法器(包括半加器、全加器)，以及组合电路的实际应用；
3. 学习如何利用 Logisim ， 通过真值表和表达式两种方式生成电路。

二、逻辑设计

【必做内容】

题目1：编码器 Pro

要添加一个有效标识变量 en ， 只要给 en 赋初值 1，当输入为有效的 4 位数时，不改变 en，而当输入为其他的四位数时，则修改 en 为 0 ， 对应模块如下：

```
module encode(
    input [3:0]         I,
    output reg [1:0]    Y,
    output reg          en
);
    always @(*) begin
        en = 1;
        case (I)
            4'b1000: Y = 2'b11;
            4'b0100: Y = 2'b10;
            4'b0010: Y = 2'b01;
            4'b0001: Y = 2'b00;
            default: begin
                Y = 2'b00;
                en = 0;
            end
        endcase
    end
endmodule
```

题目2：2bits 半加器

根据真值表，可利用卡诺图进行化简，然后便得出对应的逻辑表达式，实验模块代码如下：

```
module adder2bit(
    input           [1:0]         a,
    input           [1:0]         b,
    output   reg    [1:0]         out,
```

```
    output   reg                 Cout
);
    always @(*) begin
        Cout = (a[1]&b[1]) | (a[0]&b[1]&b[0]) | (a[1]&a[0]&b[0]);
        out[1] = (~a[1]&a[0]&~b[1]&b[0]) | (a[1]&a[0]&b[1]&b[0]) |
(a[1]&~b[1]&~b[0]) | (a[1]&~a[0]&~b[1]) | (~a[1]&~a[0]&b[1]) | (~a[1]&b[1]&~b[0]);
        out[0] = a[0] ^ b[0];
    end
endmodule
```

【选择性必做部分】

题目2：8bits 5的倍数检测器

将 8 位数看作四个四进制数 1-2，3-4，5-6，7-8，注意到 1-2，5-6 模 5 余 1 ，而 3-4，7-8 模 5 余 -1，那么只需要利用前面的半加器将 1-2，5-6 这两个数相加得到一个三位数，再将 3-4，7-8 这两个数相加得到另一个三位数，再判断这两个 3 位数是否模 5 同余即可。具体代码如下：

```
module multiple5(
    input          [7:0]          num,
    output   reg                  ismultiple5
);

wire [2:0]  sum1, sum2;
wire        Cout1, Cout2;
wire [1:0]  out1, out2;
reg  [2:0]  substract;

adder2bit adder2bit1(
    .a(num[1:0]),
    .b(num[5:4]),
    .out(out1),
    .Cout(Cout1)
);

adder2bit adder2bit2(
    .a(num[3:2]),
    .b(num[7:6]),
    .out(out2),
    .Cout(Cout2)
);

assign sum1[2:0] = {Cout1, out1};
assign sum2[2:0] = {Cout2, out2};

always @(*) begin
    substract = 3'b111;
    ismultiple5 = 1'b0;

    if (sum1 > sum2)
        substract = ~(sum2 + ~sum1 + 1) + 1;
```

```
    else
        substract = ~(sum1 + ~sum2 + 1) + 1;

    if (substract == 3'b101 || substract == 3'b000)
        ismultiple5 = 1'b1;
end

endmodule
```

三、测试结果与分析

【必做部分】

题目1的 top 模块代码以及 FPGA 验证结果如下：

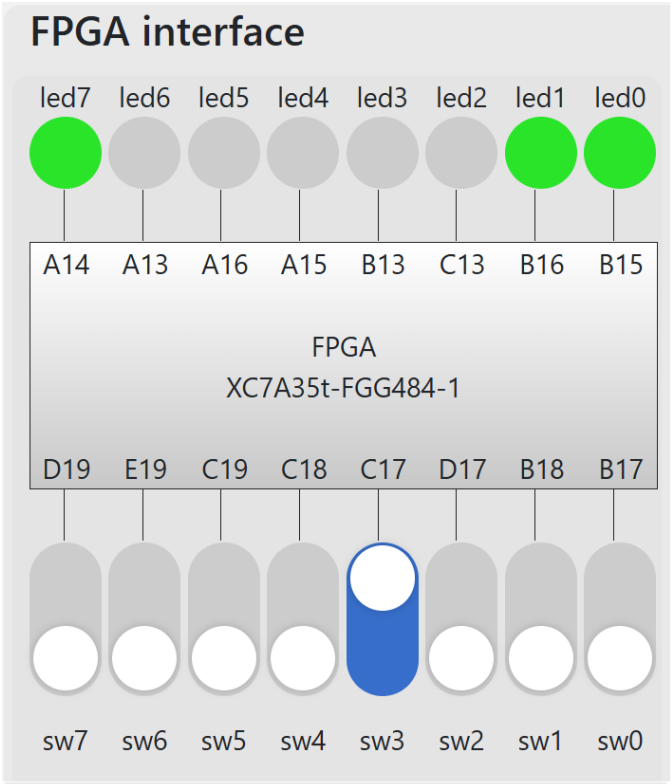
```
module top4_1(
    input [3:0]         sw,
    output [7:0]        led
);

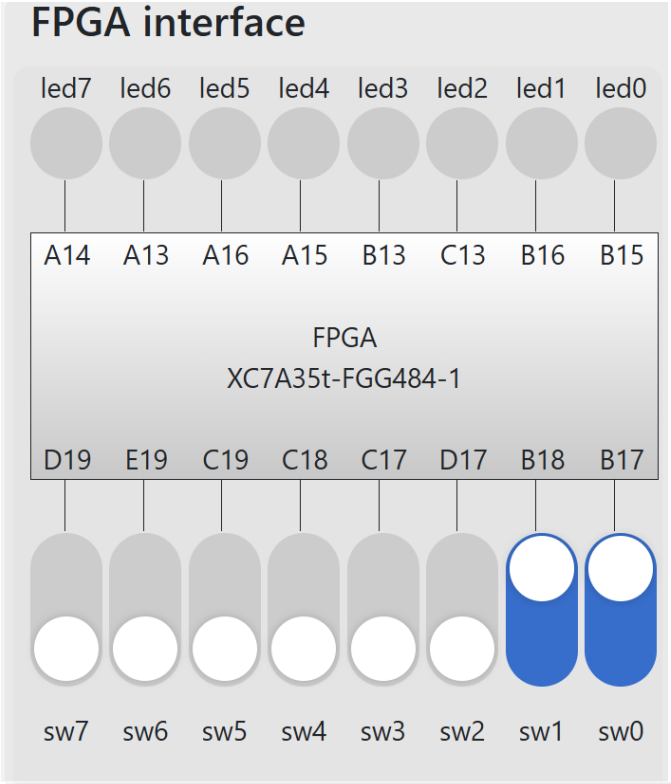
wire [1:0]  Y;
wire en;

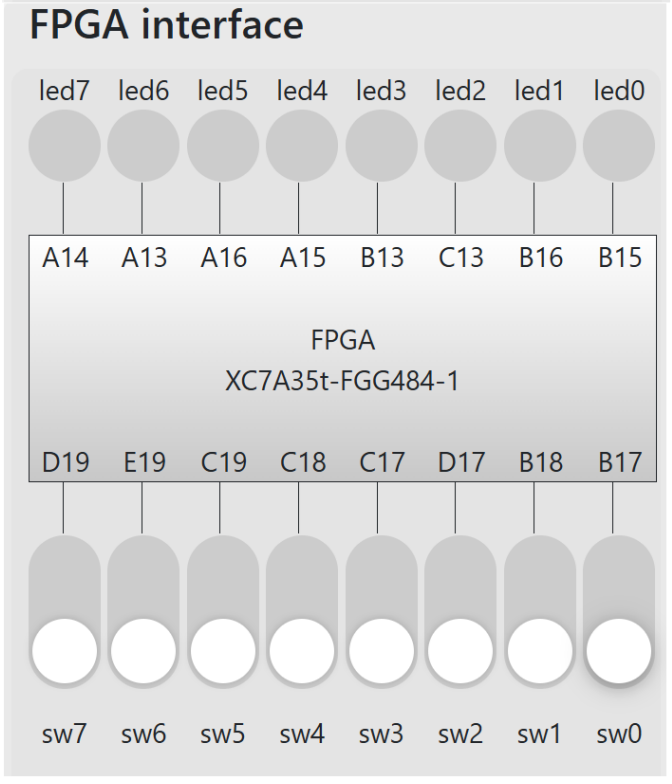
encode encode1(
    .I(sw[3:0]),
    .Y(Y),
    .en(en)
);

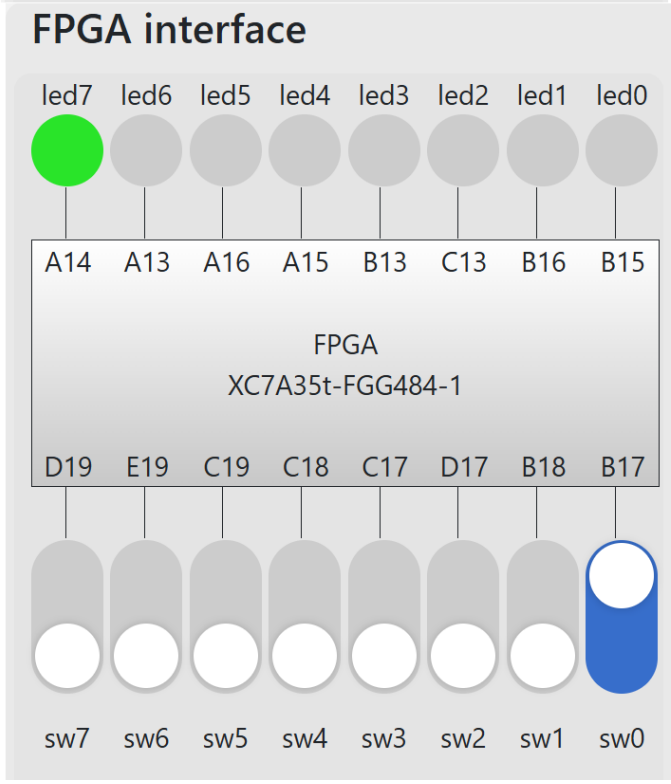
assign led = {en, 5'b00000, Y};

endmodule
```









- 如左上的图，当输入为1000时，led0，led1，led7亮，表明此时输出为 11 且有效。
- 如右上的图，输入为0011时，没有灯亮，代表输出为 00 但无效。
- 对比左下和右下的图，可知当输入为 0000 时，没有灯亮，表明输出为 00 但无效；而当输入为 0001 时，led7亮，表明输出为 00 且有效，这就区分了 0000 和 0001的情况，符合题目要求。

题目2的 top 模块如下：

```
module top2(
    input   [3:0]    sw,
    output  [7:0]    led
```

```
);

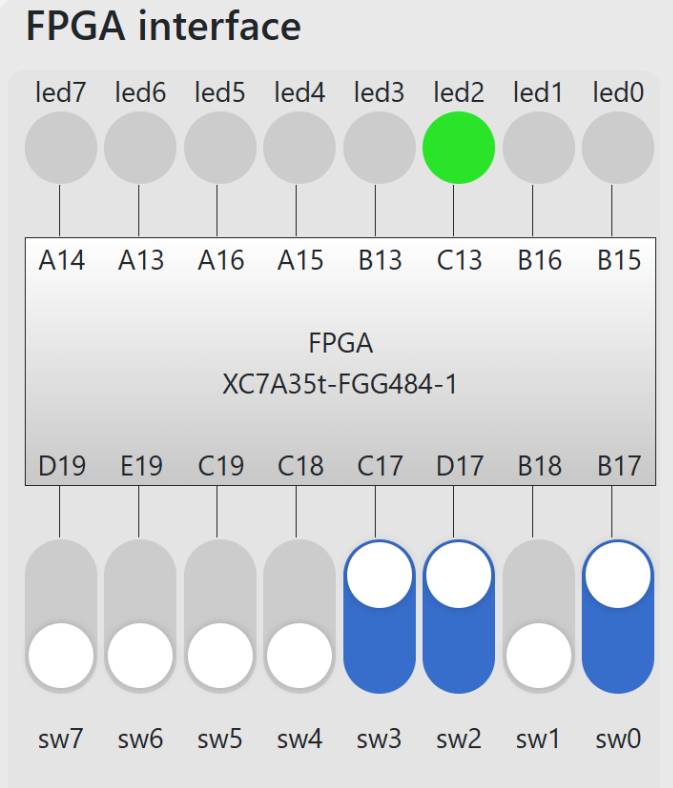
wire [1:0] out;
wire Cout;

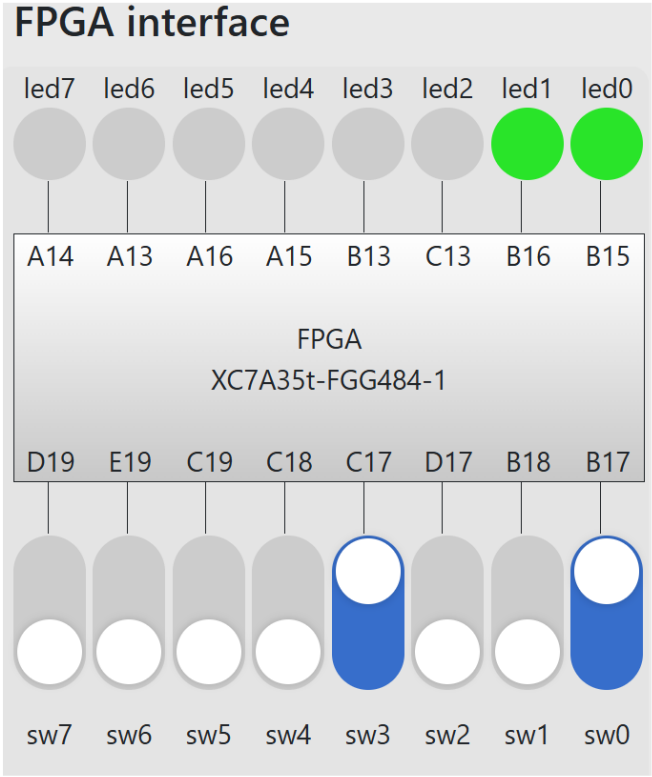
adder2bit Adder2bit(
    .a(sw[3:2]),
    .b(sw[1:0]),
    .out(out),
    .Cout(Cout)
);

assign led = {5'b00000, Cout, out};

endmodule
```

上板结果如下，这里只随机截取两个输出结果：





可见输入为11 和 01时，输出为100，输入为10和01时，输出为011，均符合半加器的要求。

【选择性必做部分】

题目2的 top 模块以及上板结果如下：

```
module top4_3(
    input   [7:0]       sw,
    output  [1:0]       led
);

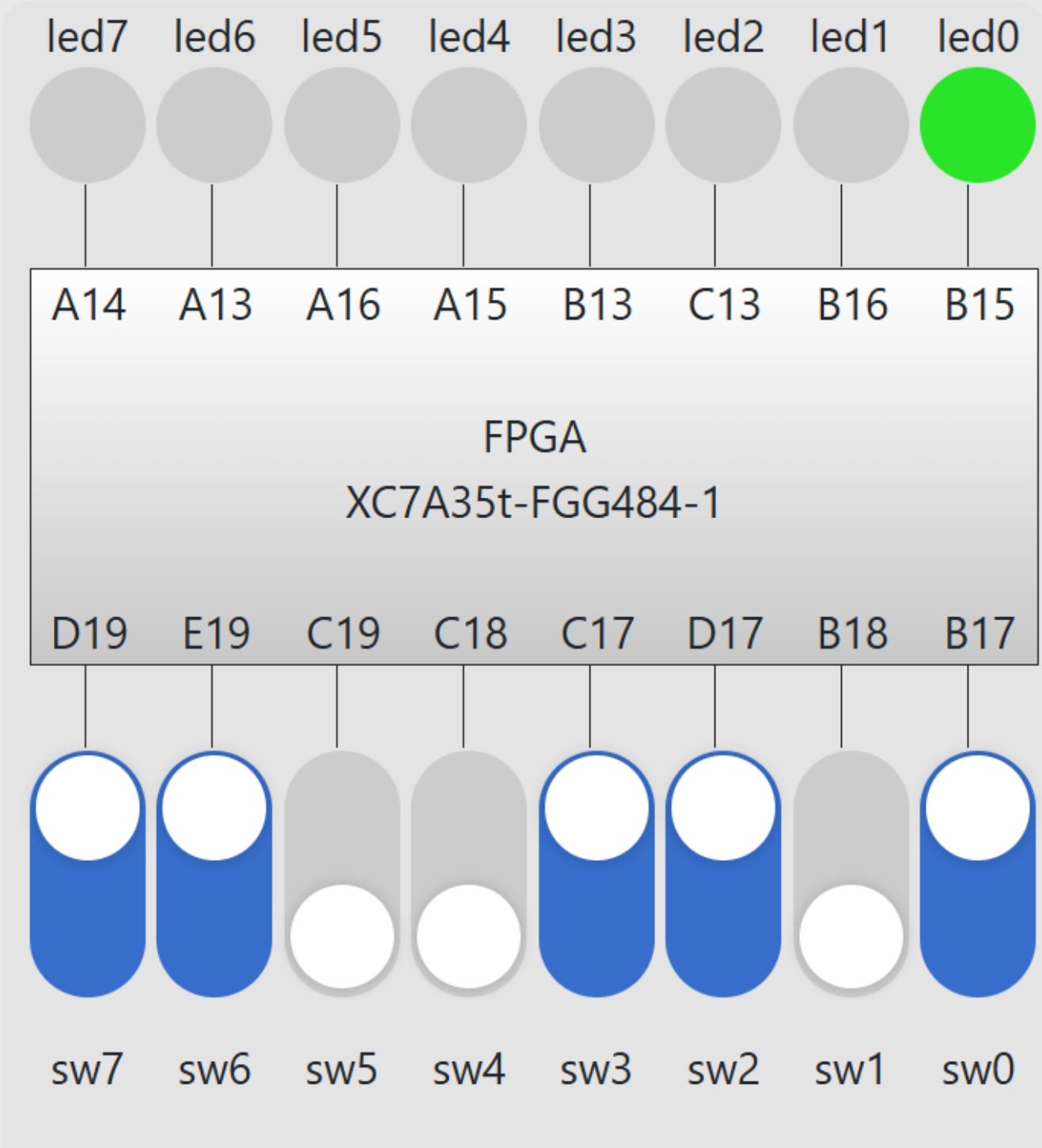
wire ismultiple5;
```

```
multiple5 Multiple5(
    .num(sw),
    .ismultiple5(ismultiple5)
);

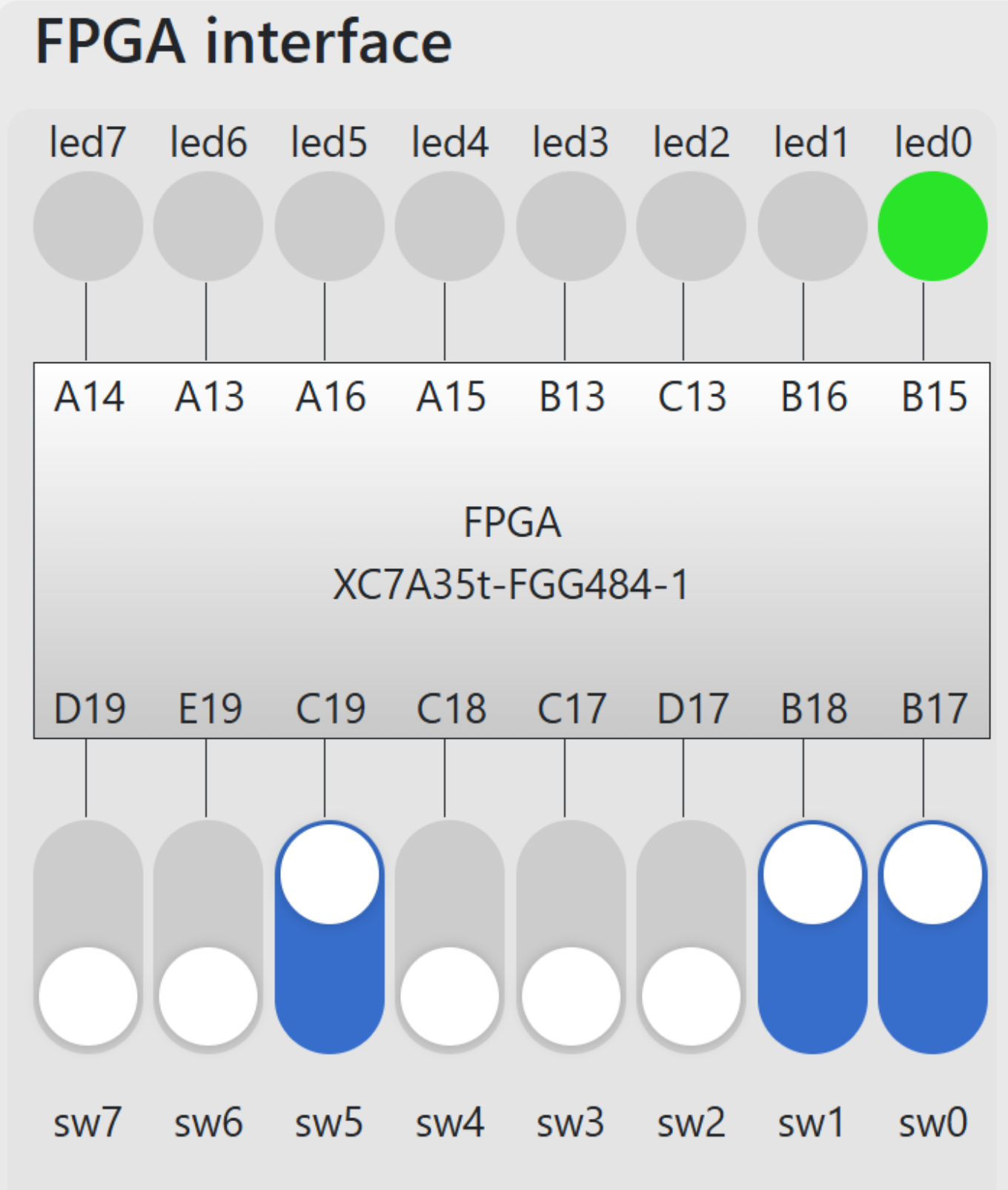
assign led = {1'b0, ismultiple5};

endmodule
```

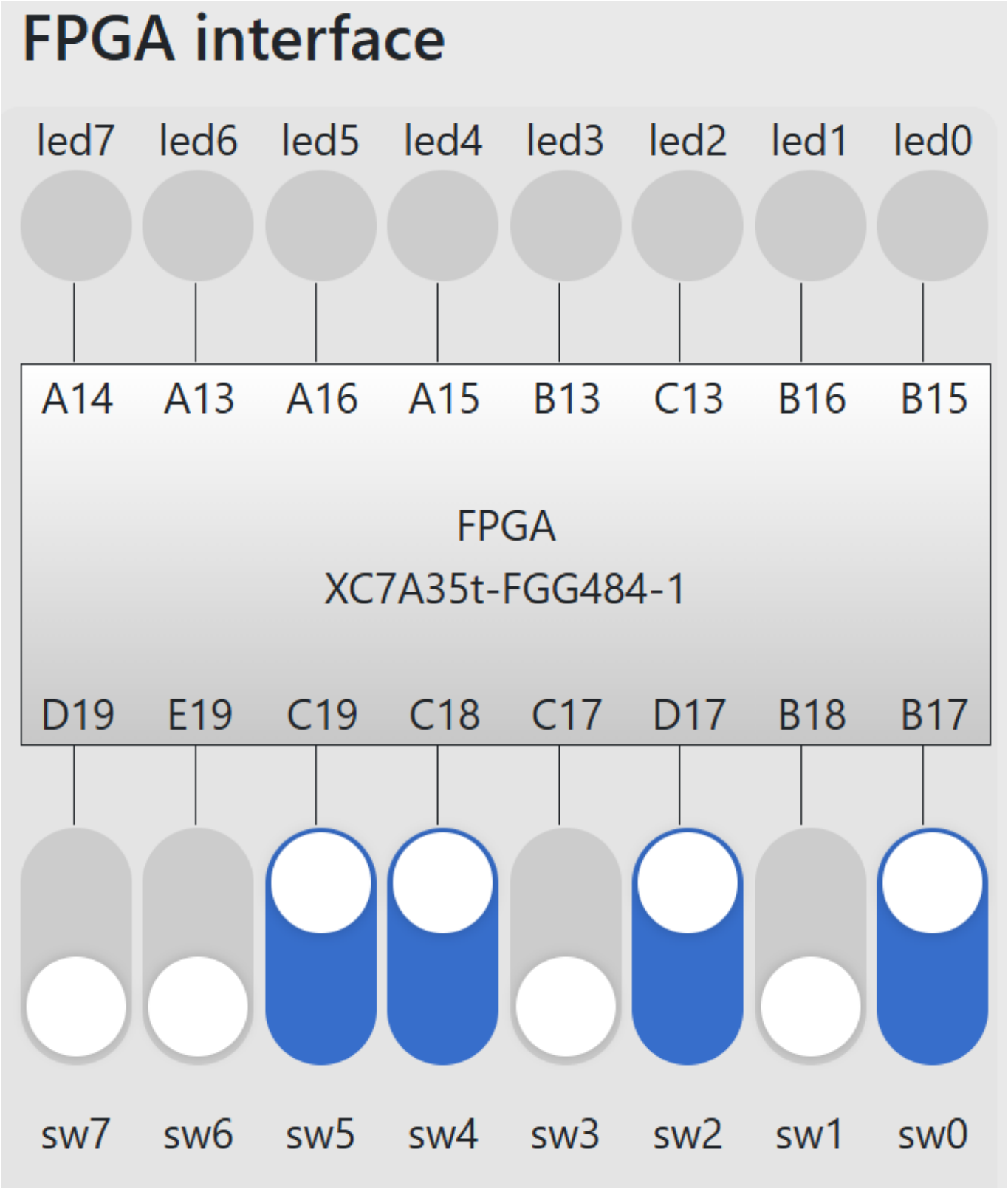
FPGA interface



由该图，输入为11001101时，1-2和5-6两位数之和为1，而3-4和7-8两位数之和为6，6 - 1 = 5，表明该数是 5 的倍数，所以 led0 亮，而将之转换为十进制数是 205，是 5 的倍数，结果正确。



同样对该图，输入为00100011时，1-2和5-6两位数之和为5，而3-4和7-8两位数之和为0，5 - 0 = 5，表明该数是 5 的倍数，所以 led0 亮，而将之转换为十进制数是 35，是 5 的倍数，结果正确。



同样对该图，输入为00110101时，1-2和5-6两位数之和为4，而3-4和7-8两位数之和为1，4 - 1 = 3，表明该数不是 5 的倍数，所以 led0 不亮，而将之转换为十进制数是 53，不是 5 的倍数，结果正确。

四、总结

通过本次实验，我对组合逻辑电路有了初步了解，认识了编码器、译码器、加法器等组合逻辑元件，会设计比较简单的组合逻辑电路，同时也加强了模块化的思想，懂得利用自己已经设计好的模块，来简化其他模块。